

Blueprints & Building Blocks: Architectural Patterns vs Design Patterns in Python

PyCon JP 2026 · August 21–22, 2026 · International Conference Center Hiroshima

Shivam Chaurasia · Solution Architect 1 · EPAM Systems, India

1. Submission Metadata

Title	Blueprints & Building Blocks: Architectural Patterns vs Design Patterns in Python
Speaker	Shivam Chaurasia, Solution Architect 1, EPAM Systems, India
Category	Python Core and Ecosystem
Audience Level	Intermediate — Basic Python class/function knowledge required. No prior pattern knowledge assumed.
Duration	30 minutes (standard slot)
45-min option	Yes — can accommodate a 45-minute format if requested
Language	English (slides will include Japanese key terms)

2. Pretalx Form Fields

Abstract

Design patterns and architectural patterns are both tools for better software—but they operate at entirely different scales. Confusing the two causes over-engineered code and under-structured systems.

Design patterns (Strategy, Observer, Decorator, Command) are micro-level—they answer 'how do I wire these objects together?' Architectural patterns (Hexagonal, CQRS) are macro-level—they answer 'how should this entire system be structured?'

In 30 minutes you will: understand the precise difference using a clear mental model; implement four design patterns idiomatically in Python 3.12+; see Hexagonal Architecture and CQRS in a real project; and leave with a two-question decision framework. All examples use Python 3.12+ with no external dependencies.

Your Experience With This Topic

As a Solution Architect at EPAM Systems with over eleven years of Python experience, I have conducted architectural assessments and led design workshops across multiple enterprise client engagements. On every team, the same confusion appeared: developers applied GoF patterns to system-level problems, or reached for Hexagonal Architecture when a Strategy pattern would have solved it in ten lines. The two-altitude mental model in this talk was forged in those real architecture reviews.

What Discussions Can You Have With Attendees?

I want to explore: When did you last reach for a pattern and later realise it was the wrong level? How do Python's first-class functions change how you think about classic GoF patterns? What does your team's actual architectural decision process look like? These questions rarely have one answer—discussing them with Python practitioners from Japan and around the world is exactly what a blog post cannot replicate.

3. Talk Outline (30 Minutes)

Time	Section	Content Note
0:00–3:00	Section 1 — The Confusion is Real	Opening hook + audience poll + thesis
3:00–7:00	Section 2 — The Three Altitudes	Mental model: function → class → module → system
7:00–9:30	Section 3a — Strategy Pattern	Replace if/elif chain with a callable; Python functions as strategies
9:30–12:00	Section 3b — Observer / Event Bus	Frozen dataclass events; Protocol-typed handlers; 15-line bus
12:00–14:00	Section 3c — Decorator Pattern	@retry in 12 lines; Python syntax IS the pattern
14:00–17:00	Section 3d — Command Pattern	Frozen dataclass commands; CommandBus + audit trail
17:00–21:30	Section 4a — Hexagonal Architecture	Ports (Protocol) + Adapters; UserService tested with InMemoryRepo
21:30–25:00	Section 4b — CQRS	Split write model / read projections; same DB, different shapes
25:00–28:00	Section 5 — Interaction Layer + Decision Framework	Patterns inside architectures; two-question decision framework
28:00–30:00	Section 6 — Closing + Discussion Invite	Takeaway; QR code; hallway discussion prompt

4. Detailed Section Notes

Section 1 — The Confusion is Real [0:00–3:00]

- Opening hook: a real GitHub issue where a developer adds full Hexagonal Architecture to a 200-line script.
- Quick audience poll: 'Who has used a GoF pattern?' / 'Who has used Hexagonal Architecture?'

- Thesis: "Same word — 'pattern' — two completely different altitudes."

Section 2 — Mental Model: The Three Altitudes [3:00–7:00]

- Visual: a zoom dial — function → class → module → service → system.
- Design patterns live at class/module level: 'How do I wire these objects together?'
- Architectural patterns live at application/system level: 'How do I structure this system so it can change, scale, and be tested?'
- Key rule: scope is the deciding factor, not complexity.
- Quick table: GoF families (Creational / Structural / Behavioural) vs Architectural patterns (Layered / Hexagonal / CQRS).

Section 3 — Design Patterns in Python [7:00–17:00]

3a Strategy — replace conditionals with callables [7:00–9:30]

- **Problem:** Problem: if payment_type == 'card': ... elif payment_type == 'crypto': ...
- **Solution:** Solution: pass a callable strategy; swap at runtime.
- **Python twist:** Python twist: plain functions are strategies — no abstract class needed.
- Live code: process_payment(charge_fn, amount) — one line replaces the if/elif chain.

3b Observer / Event Bus [9:30–12:00]

- **Problem:** order.place() directly calls inventory, billing, email — tightly coupled.
- **Solution:** Emit a frozen dataclass event; subscribers react independently.
- **Python twist:** Protocol-typed handlers; zero inheritance required.
- Live code: 15-line event bus; adding a new subscriber requires zero changes to OrderService.

3c Decorator [12:00–14:00]

- **Problem:** Retry, logging, auth duplicated or scattered.
- **Python twist:** @decorator syntax is the Decorator pattern — no wrapper class needed.
- Live code: @retry(times=3) wrapping an HTTP client in 12 lines.

3d Command [14:00–17:00]

- **Problem:** Audit trails, undo/redo, async task queues.
- **Solution:** Encapsulate requests as immutable objects.
- **Python twist:** @dataclass(frozen=True) = free immutable command object.
- Live code: CommandBus.dispatch() + audit log populated automatically.

Section 4 — Architectural Patterns in Python [17:00–25:00]

4a Hexagonal Architecture (Ports & Adapters) [17:00–21:30]

- Your domain should not know it lives inside a web framework.
- Port = interface the domain defines (UserRepository as Protocol).
- Adapter = concrete implementation (SqlAlchemyRepo, InMemoryRepo for tests).
- Live code: UserRegistrationService tested with InMemoryRepo — zero framework imports.
- Show: same service wired to FastAPI in 5 lines.
- Bridge: Repository design pattern enables this architecture.

4b CQRS — Command Query Responsibility Segregation [21:30–25:00]

- **Problem:** Reads and writes have different performance and consistency needs.

- **Solution:** Separate write model (commands) from read model (projections).
- Live code: ArticleCommandHandler (write) + ArticleQueryHandler (read) — different models, same DB.
- Payoff: read projections can be cached or denormalised without touching write logic.

Section 5 — Interaction Layer + Decision Framework [25:00–28:00]

Design patterns are the vocabulary used inside architectural boundaries.

Architecture	Design Patterns Inside
Hexagonal Architecture	Repository, Adapter, Factory
CQRS	Command, Observer, Decorator
Layered Architecture	Facade, Template Method

The Two-Question Decision Framework:

- Is my problem about how objects / functions collaborate inside one boundary? → Design pattern.
- Is my problem about how layers, services, or subsystems connect? → Architectural pattern.

Smell test: touching > 1 module → probably architectural. Contained in a single class → design pattern.

Section 6 — Closing + Open Discussion Invite [28:00–30:00]

- Recap: three altitudes, four design patterns, two architectural patterns.
- Key message: "You don't need to memorise every pattern — you need to know which altitude you're flying at."
- QR code → companion repository.
- Discussion prompt: "When did you last reach for a pattern and later realise it was the wrong level?"

5. Code Samples (Live Demos)

5.1 Strategy Pattern — callable strategy, no abstract class

```
from decimal import Decimal
from typing import Callable

# BEFORE — conditional explosion
def process_before(payment_type: str, amount: Decimal) -> None:
    if payment_type == "card":
        print(f"Charging card: ${amount}")
    elif payment_type == "crypto":
        print(f"Broadcasting crypto tx: ${amount}")
    # Every new type = another elif here AND in every caller

# AFTER — Strategy pattern (Python twist: functions are strategies)
def process(charge_fn: Callable[[Decimal], None], amount: Decimal) -> None:
    charge_fn(amount)

def charge_card(amount: Decimal) -> None:
    print(f"[Card] Charging: ${amount}")

def charge_crypto(amount: Decimal) -> None:
    print(f"[Crypto] Broadcasting: ${amount}")

# Adding a new payment method requires ZERO changes to process()
```

```
process(charge_card, Decimal("49.99"))
process(charge_crypto, Decimal("49.99"))
```

5.2 Observer / Event Bus — Protocol-typed, zero-inheritance

```
from dataclasses import dataclass
from collections import defaultdict
from typing import Callable, Any

@dataclass(frozen=True)
class OrderPlaced:
    order_id: int
    user_email: str
    total: float

EventHandler = Callable[[Any], None]

class EventBus:
    def __init__(self) -> None:
        self._handlers: dict[type, list[EventHandler]] = defaultdict(list)

    def subscribe(self, event_type: type, handler: EventHandler) -> None:
        self._handlers[event_type].append(handler)

    def publish(self, event: object) -> None:
        for handler in self._handlers[type(event)]:
            handler(event)

bus = EventBus()
bus.subscribe(OrderPlaced, lambda e: print(f"[Email]      → {e.user_email}"))
bus.subscribe(OrderPlaced, lambda e: print(f"[Inventory] order {e.order_id}"))

# OrderService knows nothing about its subscribers
bus.publish(OrderPlaced(order_id=1001, user_email="alice@example.com", total=79.99))
```

5.3 Decorator Pattern — @retry in 12 lines

```
import functools, time
from typing import Callable, TypeVar, ParamSpec

P = ParamSpec("P")
R = TypeVar("R")

def retry(*, times: int = 3, delay: float = 1.0):
    def decorator(fn: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for attempt in range(1, times + 1):
                try:
                    return fn(*args, **kwargs)
                except Exception:
                    if attempt == times:
                        raise
                    time.sleep(delay)
            return wrapper # type: ignore
        return decorator
```

```

@retry(times=3, delay=0.5)
def fetch_user(user_id: int) -> dict:
    ... # real HTTP call here

```

5.4 Command Pattern — frozen dataclass + audit-trail bus

```

from dataclasses import dataclass, field
from datetime import datetime, timezone
import uuid

@dataclass(frozen=True)
class CreateUser:
    username: str
    email: str
    command_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    issued_at: datetime = field(default_factory=lambda: datetime.now(timezone.utc))

@dataclass(frozen=True)
class DeactivateUser:
    user_id: int
    reason: str
    command_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    issued_at: datetime = field(default_factory=lambda: datetime.now(timezone.utc))

@dataclass
class CommandBus:
    _handlers: dict = field(default_factory=dict)
    _audit_log: list = field(default_factory=list)

    def register(self, command_type, handler) -> None:
        self._handlers[command_type] = handler

    def dispatch(self, command) -> None:
        self._audit_log.append(command) # automatic audit trail
        self._handlers[type(command)].handle(command)

```

5.5 Hexagonal Architecture — Port as Protocol, Adapter swap

```

from typing import Protocol, Optional
from dataclasses import dataclass

@dataclass
class User:
    username: str; email: str; id: Optional[int] = None

# PORT — owned by the domain; no imports from infrastructure
class UserRepository(Protocol):
    def get_by_email(self, email: str) -> Optional[User]: ...
    def save(self, user: User) -> User: ...

# ADAPTER 1 — production (SQLAlchemy / PostgreSQL)
class SQLAlchemyUserRepo:
    def get_by_email(self, email: str) -> Optional[User]: ... # real DB query
    def save(self, user: User) -> User: ... # real DB insert

```

```
# ADAPTER 2 – tests (in-memory dict, zero setup)
class InMemoryUserRepo:
    def __init__(self): self._store: dict[str, User] = {}
    def get_by_email(self, email: str) -> Optional[User]:
        return self._store.get(email)
    def save(self, user: User) -> User:
        self._store[user.email] = user; return user

# APPLICATION – depends only on the Port, not on any Adapter
@dataclass
class UserRegistrationService:
    repo: UserRepository          # inject SQLAlchemy in prod; InMemory in tests

    def register(self, username: str, email: str) -> User:
        if self.repo.get_by_email(email):
            raise ValueError(f"Email already registered: {email}")
        return self.repo.save(User(username=username, email=email))
```

5.6 CQRS — separate write model and read projection

```
from dataclasses import dataclass, field
import uuid

# WRITE SIDE – enforces invariants
@dataclass
class ArticleAggregate:
    id: str; title: str; body: str; published: bool = False

    def publish(self) -> None:
        if self.published: raise ValueError("Already published")
        self.published = True

    def update_body(self, new_body: str) -> None:
        if self.published: raise ValueError("Cannot edit published article")
        self.body = new_body

# READ SIDE – flat projection, optimised for queries
@dataclass(frozen=True)
class ArticleSummary:
    id: str; title: str; published: bool    # no body – list view doesn't need it

@dataclass(frozen=True)
class ArticleDetail:
    id: str; title: str; body: str; published: bool    # full detail view

# The same DB, two different shapes – reads never go through the write model
```

6. Slide Structure (24 Slides)

#	Slide Title	Content Note
1	Title Slide	Title, speaker name + title, EPAM Systems, PyCon JP 2026, QR code
2	The Expensive GitHub Issue	Screenshot of a developer adding

		Hexagonal Architecture to 200-line script
3	Audience Poll	'Who has used a GoF pattern?' / 'Who has used Hexagonal Architecture?'
4	The Three Altitudes (Visual)	Zoom dial: function → class → module → service → system
5	Design Patterns — Quick Map	Table: Creational / Structural / Behavioural with examples
6	Architectural Patterns — Quick Map	Table: Layered, Hexagonal, CQRS, Event-Driven
7	Pattern 1: Strategy — Problem	Before code: the if/elif chain
8	Pattern 1: Strategy — Solution	After code: callable strategy + Python twist callout
9	Pattern 2: Observer — Problem	OrderService coupled to 3 systems
10	Pattern 2: Observer — Solution	EventBus code + Protocol-typed handlers
11	Pattern 3: Decorator	@retry code; 'Python syntax IS the pattern' callout
12	Pattern 4: Command	Frozen dataclass command + CommandBus dispatch + audit log
13	Design Patterns Summary Table	4 patterns: problem / solution / Python idiom
14	Pattern 5: Hexagonal — Concept	Port & Adapter diagram: hexagon with plugs
15	Pattern 5: Hexagonal — Code	UserRepository Protocol + InMemoryRepo + UserRegistrationService
16	Pattern 5: Hexagonal — Test Payoff	pytest test with InMemoryRepo — zero framework imports
17	Pattern 6: CQRS — Concept	Diagram: write model → DB ← read projection
18	Pattern 6: CQRS — Code	ArticleAggregate vs ArticleSummary / ArticleDetail
19	The Interaction Layer	Table: architecture → design patterns inside it
20	Decision Framework	Two questions + smell test — large text, easy to photograph
21	Anti-Patterns to Avoid	Singleton overuse; Hexagonal for scripts; CQRS without need
22	Python Superpowers Recap	Callables, Protocol, frozen dataclass, @decorator syntax
23	Resources	3 books + 2 links; QR code to companion repo
24	Q&A + Discussion Prompt	'When did you last reach for the wrong level?'

7. Review Criteria Self-Assessment (PyCon JP 2026)

Criterion	How this proposal addresses it
Firsthand experience	Shivam has conducted architectural assessments and

	led design workshops at multiple EPAM enterprise client engagements. The two-altitude confusion is a pattern he has observed and resolved repeatedly — not a theory borrowed from textbooks.
Originality	The talk's core value is the two-question decision framework and the three-altitude mental model — a synthesis not found in existing GoF or architecture documentation. Code examples show Python-idiomatic implementations that collapse classic boilerplate.
Clarity	Six named sections, each time-boxed. Each design pattern gets one problem, one solution, one Python-specific twist, and one live code demo. The decision framework is a single slide attendees can photograph.
Interactivity	Closes with an explicit discussion invite. The central question — 'When did you last reach for the wrong level?' — is one every practitioner can answer differently, making the hallway conversation richer in an international conference setting.
Relevance	Python's flexibility makes it uniquely prone to this confusion. With Django, FastAPI, and asyncio-based architectures increasingly common in Japanese enterprise software, the decision framework is immediately applicable.

8. Patterns Overview at a Glance

Design Patterns (class / module level)

Pattern	Problem Solved	Python Idiom
Strategy	Conditional explosion	Callable / Protocol — no abstract class needed
Observer	Tight coupling on events	Frozen dataclass events + EventBus
Decorator	Cross-cutting concerns scattered	@decorator syntax is the pattern
Command	Audit trail / undo / async queues	frozen=True dataclass as immutable command

Architectural Patterns (application / system level)

Pattern	Problem Solved	Python Approach
Hexagonal / Ports & Adapters	Domain coupled to framework/DB	Protocol as Port; InMemoryRepo for tests
CQRS	Reads and writes share one rigid model	Separate Aggregate (write) + Projection (read)
Layered	Fat views; no separation of concerns	presentation / application / domain / infra

9. References & Further Reading

Design Patterns (GoF)	Gamma, Helm, Johnson, Vlissides — Addison-Wesley 1994
Architecture Patterns with Python	Harry Percival & Bob Gregory — O'Reilly 2020
Clean Architecture	Robert C. Martin — Prentice Hall 2017
Designing Data-Intensive Applications	Martin Kleppmann — O'Reilly 2017
Building Microservices (2nd Ed.)	Sam Newman — O'Reilly 2021
Python Protocols (PEP 544)	docs.python.org/3/library/typing.html#typing.Protocol
Python dataclasses	docs.python.org/3/library/dataclasses.html